Project Proposal: Blood Donor Management System

Project Title: Blood Donor Management System
Prepared by: Sehla Razzak(24k-0575), Hiba Faiq(24k-0781), Tania(24k-0713)
Date: 6th May

1. Introduction & Problem Statement

Blood banks currently face significant challenges in managing donor information, tracking blood inventory in real-time, and processing hospital requests efficiently. Existing manual or fragmented systems often lead to:

- **Blood Wastage:** Units expiring due to poor tracking.
- **Delayed Responses:** Hospitals waiting too long for urgent blood requests.
- **Data Redundancy:** Duplicate donor records and lack of donor eligibility tracking.
- **Lack of Centralization:** No single source of truth for blood stock levels, donation camps, and user roles (admins, lab techs, hospitals).

To address these issues, we propose the development of a **Blood Donor Management System (BDMS)** – a comprehensive, database-driven web application designed to streamline blood bank operations and ultimately help save lives by ensuring timely blood availability.

2. Project Objectives

The primary objectives of this project are:

1. **Centralize Data:** Create a unified Oracle database to manage donors, blood inventory, hospital requests, and donation camps.
2. **Automate Tracking:** Automatically monitor blood stock levels, expiry dates (Red = Expired, Yellow = Quarantined, Green = Available), and safety status.

3. **Enable Real-time Requests:** Allow hospitals to submit blood requests (by blood group, quantity, urgency) and allow admins to fulfill them with one click.
4. **Provide Analytics:** Display real-time statistics (Total Donors, Available Bags, Pending Requests, Upcoming Camps) on an admin dashboard.
5. **Ensure Data Integrity:** Use sequences, triggers, foreign keys, and check constraints to maintain data accuracy.

3. Proposed System Architecture & Technologies

Based on the requirements, the following technology stack is proposed (matching your finalized report):

Layer	Technology	Purpose
Backend	Python with Flask Framework	Handle business logic, API endpoints, and server-side processing.
Database	Oracle Database (with oracle-db connector)	Store all schemas, enforce relationships, and manage transactions.
Frontend	HTML5, CSS3, JavaScript (Vanilla JS)	Provide a responsive, single-page interface for users.
Key Features	Sequences, Triggers, Joins, Constraints	Auto-increment IDs, log admin actions, ensure data validity.

4. Scope of Work & Deliverables

4.1 Database Schema Design (Deliverable 1)

- **Users & Roles Table:** Parent table (`Users`) with child tables (`Donor`, `Hospital`, `Employee`, `Admin`, `LabTech`) using inheritance.
- **Inventory Table:** `blood_inventory` storing blood group, quantity (ml), expiry date, status.

- **Operations Tables:** `DonationRecord`, `Appointment`, `BloodRequest`, `DonationCamp`.
- **Supporting Tables:** `Notification`, `System_log`, `Blood_report`.
- **Automation:** Implement sequences and "BEFORE INSERT" triggers for all primary keys.

4.2 Backend API Endpoints (Deliverable 2)

Build REST APIs to serve data to the frontend, including:

- `GET /api/dashboard/stats` – for real-time metrics.
- `GET /api/donors` – list all donors.
- `POST /api/requests` – for hospitals to create requests.
- `PUT /api/requests/<id>/fulfill` – approve and fulfill requests.

4.3 Frontend User Interface (Deliverable 3)

A responsive single-page application featuring:

- **Navigation Bar:** Dashboard, Donors, Blood Stock, Requests, Camps, Register Donor, New Request.
- **Dashboard Cards:** Showing Total Donors, Total Blood Bags (calculated as `SUM(quantityML)/450`), Pending Requests, Upcoming Camps.
- **Data Tables:** Sortable tables with color-coded badges (Green, Yellow, Red).
- **Action Buttons:** One-click "Fulfill" for pending requests.

5. Functionality Map

Module	Key Functions

Donor Management	View donor list, register new donors, auto-generate UserID, check medical history & eligibility.
Blood Inventory	View stock (by blood group), color-coded expiry flags, safety status (isSafe flag).
Request Management	Hospitals submit requests (blood group, quantity, urgency); Admin updates status (Pending → Approved → Fulfilled/Rejected).
Camp Management	View upcoming/past camps (name, location, start/end date, organizer).
Logging & Security	Track admin actions in `System_log`; Unique email constraint to prevent duplicates.

6. Development Methodology & Timeline

We will use an **Agile (Iterative)** approach. The estimated timeline is **6-8 weeks**:

Phase	Activities	Duration
Phase 1: Planning & Schema	Design ER diagram, define tables, sequences, and triggers.	1 week
Phase 2: Backend Setup	Setup Flask app, Oracle connection, build API endpoints.	2 weeks
Phase 3: Frontend UI	Build HTML/CSS/JS pages, integrate APIs, and implement dashboard cards.	2 weeks
Phase 4: Integration & Testing	Test donor registration, request flow, inventory expiry logic, edge cases.	1.5 weeks
Phase 5: Documentation	Finalize user manual, technical report, and deployment guide.	0.5 week

7. Testing & Validation Plan

To ensure reliability, the system will be tested with:

1. **Sample Data:** Insert minimum 10 users, 5 donors, 10 inventory items, 5 requests, 5 camps, and 5 logs.
2. **Flow Tests:**
    a. Donor registration (sequential ID generation via sequence).
    b. Request creation → status update → fulfillment.
    c. Inventory expiry and safety flag calculation.
3. **Constraint Validation:** Ensure foreign keys, check constraints (e.g., blood group values), and unique emails are enforced.
4. **Join Queries:** Verify that data can be retrieved by joining `Users` with `Donor` and `Hospital` tables.

8. Expected Outcomes

Upon completion, this system will successfully:

- Provide a centralized database for all blood bank operations.
- Reduce blood wastage through automated expiry tracking.
- Speed up hospital response times via real-time request fulfillment.
- Deliver a user-friendly web interface for both administrators and hospital staff.
- Serve as a scalable foundation for a real-world blood bank management solution.

9. Budget & Resource Requirements (If applicable)

- **Software:** Oracle Database (Free for development), Python/Flask (Open Source).
- **Hardware:** Standard development machine (8GB+ RAM) + Server for deployment (or localhost for demo).
- **Personnel:** 1-2 Developers (Full stack) + 1 Database Designer.

10. Conclusion

The proposed **Blood Donor Management System** directly addresses the inefficiencies of current blood bank operations. By leveraging Oracle Database for integrity, Flask for a robust backend, and a responsive JavaScript frontend, this project will deliver a practical, scalable, and life-saving tool. The approach outlined in this proposal is technically feasible and aligned with the requirements documented in the final report.